

Oscar Kalid Design System

A comprehensive agent build brief for creating a distinctive, implementation-ready system for applications, websites, dashboards, editorial reports, data stories, and polished PDF output.

WORKING TITLE: O3 - NAMING REQUIRED

OWNER	Oscar Kalid
CANONICAL DOMAIN	design.oscarkalid.com
DOCUMENT VERSION	1.0
PREPARED	3 September 2026
TARGET	Design system version 1.0

PRIMARY OUTCOME

An independent agent can follow one canonical documentation entry point and produce coherent products or evidence-led reports without inventing an undocumented visual language.

Contents

This brief is organized from project mandate through foundations, application and report modes, implementation, agent interfaces, testing, governance, and release.

Contents 2

How to use this brief 6

The master instruction 6

1. Desired outcome 7

 1.1 Primary goals 7

 1.2 Non-goals 7

2. Identity and naming 8

 2.1 Naming requirements 8

 2.2 Required identity output 8

3. Design charter 9

 3.1 Required principles 9

 3.2 Personality definition 9

 3.3 Signature moves 9

4. Discovery and visual-direction process 10

 4.1 Inspect source material 10

 4.2 Create reference artifacts before abstraction 10

 Application references 10

 Report references 10

 4.3 Compare compositions before polishing 11

5. System architecture 11

6. Foundations specification 11

 6.1 Color 11

 6.2 Typography 12

 6.3 Spacing and density 12

 6.4 Shape 13

 6.5 Elevation, surface, and depth 13

 6.6 Layout and responsive behavior 13

 6.7 Motion 14

 6.8 Iconography and imagery 14

 6.9 Content and voice 14

 6.10 Accessibility 15

 6.11 Internationalization 15

 6.12 Data visualization 15

7. Application mode	16
7.1 Application composition rules	16
7.2 Application shell variants	16
7.3 Health-product requirements	16
8. Report mode	17
8.1 Report operating principles	17
8.2 Report planning canvas	17
8.3 First-view requirements	18
8.4 Editorial hierarchy	18
8.5 Report layout	18
8.6 Evidence rules	18
8.7 Tables	19
8.8 Charts and figures	19
8.9 Calculators and interactive reports	19
8.10 Report surfaces and color	20
8.11 PDF and print specification	20
9. Component system	21
9.1 Required component families	21
Layout and structure	21
Actions	21
Inputs	21
Navigation	22
Information and surfaces	22
Feedback and state	22
Data visualization	23
Report primitives	23
9.2 Component documentation contract	23
9.3 State completeness	24
10. Patterns and templates	25
10.1 Required application patterns	25
10.2 Required report patterns	25
10.3 Pattern page contract	25
11. Extension protocol	26
Creating an undocumented component	26
12. Tokens and machine-readable contracts	27
12.1 Token hierarchy	27
12.2 Required token outputs	27
13. Implementation packages	28
13.1 Package expectations	28
Tokens package	28

Icon package	28
React reference package	28
Report package	29
Linting and validation	29
14. Documentation website	29
14.1 Information architecture	29
14.2 Documentation-page requirements	30
14.3 Technical accessibility for agents	30
15. Agent interface	30
15.1 Required llms.txt	30
15.2 llms-full.txt	31
15.3 Machine manifest	31
15.4 Repository adapters	31
15.5 Optional agent tools	32
16. Reusable prompts	32
16.1 Application prompt	32
16.2 Report prompt	33
16.3 Audit prompt	33
17. Testing and quality assurance	33
17.1 Automated testing	33
17.2 Manual visual review	33
17.3 Agent-comprehension tests	34
17.4 Acceptance thresholds	35
18. Governance and release management	35
18.1 Decision records	35
18.2 Versioning	36
18.3 Contribution process	36
19. Execution plan	37
Phase 0 - Establish the project	37
Phase 1 - Define visual judgment	37
Phase 2 - Build foundations	37
Phase 3 - Build the core system	37
Phase 4 - Prove composition	37
Phase 5 - Publish the agent layer	38
Phase 6 - Validate and release	38
20. Required deliverables	38
Identity and governance	38
Foundations	38
Implementation	39

Documentation39

Agent interface39

Quality39

21. Definition of done 40

22. First actions for the executing agent 40

Appendix A - Fast visual inspection routine41

Appendix B - Component specification template41

Appendix C - Pattern specification template 42

Appendix D - Report preflight 42

Appendix E - Source and inspiration notes 43

How to use this brief

Paste this entire document into the primary agent responsible for building the design system. Give that agent access to the project repository, relevant visual references, and the ability to create and test files. If the environment supports long-running work, instruct the agent to continue through all phases rather than stopping after a plan.

This document is the project mandate. The agent may make reversible implementation decisions when information is missing, but it must record material assumptions and must not silently replace the objectives in this brief with a generic component library or documentation template.

The PDF titled [What decides a Vercel report.pdf](#) is a visual-quality reference for the report mode. If the agent is not operating in the original workspace, attach that PDF separately. The public Vercel report guidance at <https://vercel.com/design.md> is an additional reference for editorial restraint, evidence hierarchy, responsive report composition, and agent-facing design guidance. Use those sources as quality references, not as assets to copy or as permission to imitate Vercel branding.

Material 3 is a benchmark for completeness, clarity, composability, accessibility, and implementation depth. The new system must develop its own identity and must not present itself as Google-authored or reproduce Google brand assets.

The master instruction

You are the lead product designer, design-systems architect, information architect, design engineer, accessibility specialist, technical writer, documentation engineer, and quality owner for a new personal design system created for Oscar Kalid.

Your job is to produce the working system, not only a proposal for it. Design its visual language, formalize its rules, implement its tokens and foundational components, build its documentation website, create reference applications and reports, publish agent-readable entry points, and verify that independent agents can use the system without inventing undocumented visual values.

The system must support two related but distinct output modes:

1. Application mode: Interactive products such as health trackers, finance tools, smart-home dashboards, productivity applications, settings surfaces, mobile experiences, and operational interfaces.
2. Report mode: Editorial, analytical, research, decision, benchmark, comparison, proposal, brief, calculator, and evidence-heavy documents that can be delivered as responsive websites or polished PDFs.

Both modes must share identity, typography, color logic, tokens, accessibility standards, and quality principles. They must not use the same composition reflexively. Application mode may use persistent navigation, controls, layered surfaces, and compact repeated components. Report mode must prioritize the argument, reading order, evidence, long-form typography, tables, figures, print behavior, and editorial restraint.

Proceed through the phases in this brief. Inspect the existing repository and references before changing architecture. Preserve a host project's framework and conventions when they are suitable. When no project exists, choose a small, maintainable, standards-based implementation with a framework-independent token core and a typed reference component library.

Do not stop after generating mood boards, tokens, or a component inventory. Continue until the documented release criteria are satisfied or a genuine external blocker prevents progress. At every phase, favor working artifacts over descriptions of artifacts.

1. Desired outcome

The finished design system must let Oscar give an agent a direction such as:

Build a responsive health tracker using the Oscar Kalid design system. Begin at <https://design.oscarkalid.com/llms.txt>, use the documented tokens and components, follow the health-data and dashboard patterns, and apply the extension protocol to any app-specific components.

The agent should then be able to create a polished, recognizable result without receiving a custom visual-design prompt for every screen.

The same system must support a report request such as:

Produce an evidence-led report using the Oscar Kalid report mode. Begin at <https://design.oscarkalid.com/llms.txt>, preserve all supplied facts and qualifications, make the first view communicate the main finding, support executive and audit reading paths, and export an accessible PDF after visual verification.

Success means that separate agents, working on different product categories, produce artifacts that feel related through consistent judgment rather than through repetitive templates.

1.1 Primary goals

- Establish an original and recognizable design language owned by Oscar Kalid.
- Make high-quality visual judgment repeatable across applications and reports.
- Publish precise foundations, tokens, components, patterns, examples, and extension rules.
- Give humans and agents one canonical, stable documentation source.
- Provide implementation-ready assets instead of relying on prose alone.
- Support responsive web first while leaving a credible path to mobile-native implementations.
- Make accessibility, internationalization, themes, and reduced motion part of the system rather than later corrections.
- Make reports as considered as applications, including editorial hierarchy, data presentation, citations, tables, print, and PDF export.
- Permit domain-specific components without allowing the design language to drift.
- Establish governance so the system can evolve without silently breaking consumers.

1.2 Non-goals

- Do not attempt to design every imaginable domain component before release.
- Do not create a giant collection of unrelated UI examples with no governing rules.
- Do not make a clone of Material 3, Vercel, Apple, Linear, or another existing system.
- Do not equate consistency with making every page use the same card grid or hero layout.
- Do not rely on a Figma file, PDF, or JavaScript-only website as the sole source of truth.
- Do not let agents invent raw colors, spacing, radii, typography sizes, shadows, or interaction states when documented alternatives exist.
- Do not treat a successful build as proof of design quality; visual and semantic review are required.

2. Identity and naming

The working name `O3` must not automatically become the public name. A version number is meaningful for Material Design because it follows earlier generations. A new system should have a durable name that still works after many releases.

Use `design.oscarkalid.com` as the canonical domain during naming and after launch. This keeps documentation URLs stable if the system is renamed. If a distinctive final name is selected, a memorable alias such as `<name>.oscarkalid.com` may redirect to the canonical site.

2.1 Naming requirements

Run a bounded naming phase before finalizing brand assets. Generate candidates and evaluate them against these requirements:

- Distinctive enough to search for without being confused with a major product or model.
- Easy to pronounce after reading it once.
- Preferably one or two words and no more than three syllables.
- Appropriate for both product UI and editorial reports.
- Not dependent on a current visual trend, technology, or version number.
- Able to support phrases such as `[Name] Button`, `[Name] Report Mode`, and `[Name] Design System`.
- Compatible with package namespaces, repository names, and subdomains.
- Free of misleading affiliation with another company.
- Screenable for obvious trademark, package, repository, and search collisions before adoption.

Do not select a name merely because its domain is available. First define the system's personality and signature design principles, then select the name that expresses them.

2.2 Required identity output

Produce:

- Final name and one-sentence rationale.
- Pronunciation and capitalization.
- Full legal/descriptive form: `[Name] Design System by Oscar Kalid`.
- Short form and package prefix.
- Canonical URL and redirects.
- Wordmark treatment and minimum-size rules.
- A compact identity statement agents must repeat in metadata.
- An explicit disambiguation note for any likely naming collision.
- A short list of names rejected and the concrete reason each was rejected.

Until this phase is complete, use `[SYSTEM_NAME]` in generated files and maintain a single configuration value that can replace it across the repository.

3. Design charter

Before producing components, write and approve a concise design charter. It must establish the system's judgment, not only its appearance.

3.1 Required principles

Refine these into five to seven memorable principles:

1. Clarity before decoration. Structure, typography, and content must communicate before color or effects are added.
2. Relationships create hierarchy. Relative scale, alignment, density, and spacing should reveal what belongs together and what matters most.
3. Meaning earns emphasis. Strong color, contrast, size, motion, and surfaces are reserved for meaningful states, actions, or evidence.
4. Composition follows the reader's job. Choose the page structure from the task, content, and evidence rather than from a default template.
5. Precision creates trust. Values, units, labels, states, controls, and claims must be exact and internally consistent.
6. Adapt without losing identity. Mobile, desktop, application, and report outputs should recompose appropriately while sharing the same design DNA.
7. New parts inherit the system. Undocumented components must be built from existing primitives and tokens according to the extension protocol.

For each principle provide:

- A one-sentence rule.
- The problem it prevents.
- A correct visual example.
- An incorrect visual example.
- A fast inspection test an agent or reviewer can perform.

3.2 Personality definition

Define the system on explicit scales rather than relying on adjectives alone:

- Restrained to expressive.
- Editorial to product-like.
- Dense to spacious.
- Soft to geometric.
- Familiar to novel.
- Neutral to branded.
- Static to kinetic.
- Formal to conversational.

Choose a position and acceptable range for application mode and report mode. Record intentional differences between the modes.

3.3 Signature moves

Define three to five recognizable characteristics that create identity without becoming decorative gimmicks. A signature move may involve:

- Shape relationships.

- A particular typographic contrast.
- Distinctive navigation geometry.
- A disciplined method for layered surfaces.
- A characteristic data-label treatment.
- A recognizable relationship between large claims and supporting evidence.
- A specific transition or state change.

Every signature move must state when it is used, when it is withheld, and how it behaves in light, dark, compact, and reduced-motion contexts.

4. Discovery and visual-direction process

Do not begin with a component catalogue. Begin with representative problems.

4.1 Inspect source material

Collect and classify:

- Existing Oscar Kalid products, prototypes, screenshots, logos, and preferences.
- The supplied Material 3-style dashboard screenshot as a quality and coherence reference.
- The supplied Vercel-style report PDF as a report-quality reference.
- Products the owner considers excellent and products the owner dislikes.
- Recurring content types: metrics, timelines, navigation, tables, charts, forms, settings, alerts, summaries, long-form prose, sources, and calculations.
- Required platforms, frameworks, browsers, assistive technologies, themes, and export formats.

Do not copy the references literally. Identify the underlying qualities the owner values: systematic hierarchy, consistency, restraint, strong typography, complete component states, coherent spacing, good dark mode, clear evidence, and polished composition.

4.2 Create reference artifacts before abstraction

Design at least five reference applications and three report types before freezing version 1.0:

Application references

1. Health tracker with summary metrics, trends, goals, activity history, and data-entry flows.
2. Personal finance application with accounts, spending, subscriptions, transactions, budgets, and a dense table.
3. Smart-home dashboard with scenes, device states, environmental data, schedules, and alerts.
4. Productivity application with navigation, search, tasks, filters, detail panes, and settings.
5. Content or community application with feeds, profiles, media, comments, and moderation states.

Report references

1. Executive decision brief with one recommendation and supporting evidence.
2. Research or benchmark report with methods, charts, tables, caveats, and sources.
3. Interactive comparison or calculator that can also export a stable PDF record.

Each reference must include desktop and narrow layouts, light and dark themes where appropriate, loading and empty states, failure states, keyboard behavior, and at least one difficult edge case such as long labels, missing data, negative values, or localization expansion.

4.3 Compare compositions before polishing

For every flagship reference, explore at least two materially different compositions before selecting one. The alternatives must differ in hierarchy, information topology, evidence placement, or interaction model – not only color or decoration.

Record:

- The reader's primary job.
 - The first thing they must understand or do.
 - The dominant object or relationship.
 - Why the selected composition reduces work for the reader.
 - Which familiar layout was rejected and why.
-

5. System architecture

Organize the system into seven layers. Documentation, code, and design assets must use the same vocabulary.

1. Principles: The judgment governing all outputs.
2. Foundations: Color, type, spacing, shape, grid, elevation, motion, icons, imagery, content, accessibility, and localization.
3. Primitives: Low-level layout and interaction building blocks such as stack, cluster, grid, text, surface, divider, focus ring, and visually hidden content.
4. Components: Reusable interface units with defined anatomy, behavior, states, variants, and accessibility contracts.
5. Patterns: Repeatable compositions that solve user problems across multiple components.
6. Templates and references: Complete examples showing how the system handles real applications and reports.
7. Agent interface: `llms.txt`, Markdown pages, manifests, local instruction adapters, packages, examples, tests, and optional retrieval tools.

No layer may redefine a decision owned by a lower layer. Component tokens may reference semantic foundation tokens, but they must not introduce unrelated raw values. Templates demonstrate the system; they do not become an undocumented second system.

6. Foundations specification

Every foundation must include a human explanation, machine-readable values, usage rules, examples, anti-examples, accessibility notes, and a change policy.

6.1 Color

Define three token levels:

1. Reference tokens: Raw palette values, named by hue and step rather than by usage.
2. Semantic tokens: Roles such as surface, text, border, action, selected, success, warning, error, and data series.
3. Component tokens: Rare aliases used only when a component needs a stable semantic contract.

Required semantic families:

- Canvas and layered surfaces.
- Primary, secondary, tertiary, and inverse text.
- Subtle, default, strong, and focus borders.

- Primary and secondary actions.
- Hover, pressed, selected, disabled, dragged, and focus states.
- Informational, success, warning, destructive, and pending states.
- Data visualization series, thresholds, reference lines, and uncertainty.
- Scrims and overlays.
- Light and dark themes.
- High-contrast accommodation.

Rules:

- Never encode state or meaning through color alone.
- Test actual foreground/background pairs, not isolated palette swatches.
- Define contrast expectations for ordinary text, large text, icons, boundaries, charts, and focus indicators.
- Avoid assigning positive or negative meaning to a hue without a label or non-color cue.
- Do not let every brand color become an interface color.
- Document how user-generated and categorical colors are assigned and repeated.
- Provide a monochrome fallback for print, accessibility, and report-mode restraint.

6.2 Typography

Define:

- Primary sans family and robust system fallbacks.
- Optional display or editorial family only if it serves a clear role.
- Monospace family for code, paths, identifiers, and technical values.
- Display, page title, section, subsection, body, compact body, label, metadata, caption, numeric, and code roles.
- Font size, line height, weight, letter spacing, optical sizing, and measure for every role.
- Responsive type behavior using bounded fluid values or explicit steps.
- Tabular numerals and alignment rules for comparable data.
- Rules for capitalization, punctuation, headings, dates, units, currency, percentages, and truncation.

Typography rules:

- Equivalent peers use identical roles even when their strings differ in length.
- Rewrite or recompose before shrinking individual text.
- Body text remains comfortably readable; small muted text must not become a strategy for hiding density.
- Long-form report prose has a controlled measure and deliberate paragraph rhythm.
- Application labels prioritize scannability and predictable alignment.
- Headings should communicate meaning, not merely label a document category.

6.3 Spacing and density

Create a named scale that supports fine control without arbitrary values. Include compact, standard, and comfortable density modes only if the system genuinely needs them.

Document relational spacing:

- Icon to label.
- Label to value.
- Heading to first paragraph.

- Field label to control to helper or error.
- Sibling rows.
- Content group to content group.
- Section to section.
- Page edge to content.
- Surface boundary to internal content.

Every visible gap must have one owner. Parent layout primitives should normally control gaps; child components must not add competing outer margins. Publish rules for collapsing, responsive changes, and dense tables.

6.4 Shape

Define:

- Radius scale.
- Which categories are square, softly rounded, strongly rounded, or circular.
- Rules for nested radii.
- Shape changes for selected, expanded, or dragged states.
- How shape differs between application and report modes.

Avoid assigning a rounded rectangle to every piece of content. A boundary must communicate grouping, affordance, state, or hierarchy that spacing alone cannot communicate.

6.5 Elevation, surface, and depth

Specify:

- Surface hierarchy.
- Border-first versus shadow-first contexts.
- Overlay, menu, dialog, popover, tooltip, sticky, and dragged elevations.
- Light and dark theme behavior.
- When a flat continuous canvas is preferred.
- How overlapping content preserves contrast.

Reject decorative shadows and fake physical depth. Application mode may use layered tonal surfaces. Report mode should normally use a continuous page and earn contrast fields sparingly.

6.6 Layout and responsive behavior

Define:

- Supported viewport range.
- Content maximums and minimums.
- Outer gutters.
- Grid columns for narrow, medium, and wide layouts.
- Reading-width rules.
- Application shell regions.
- Navigation transitions between rail, drawer, sidebar, top navigation, and bottom navigation.
- Container queries or component-level adaptation where appropriate.
- Safe areas, zoom, orientation, virtual keyboards, and coarse pointers.
- Rules for tables, charts, dialogs, side panels, and dense toolbars at narrow widths.

Do not merely scale desktop layouts down. Define recomposition: which regions stack, reorder, collapse, become disclosures, scroll locally, or change navigation form.

6.7 Motion

Define durations, easing, distances, interruption behavior, and reduced-motion alternatives for:

- Hover and press feedback.
- Selection and focus.
- Expansion and collapse.
- Navigation transitions.
- Dialogs, sheets, menus, and tooltips.
- Loading and progress.
- Reordering and drag operations.
- Data changes.

Motion must explain change, preserve continuity, or confirm action. Never gate reading behind animation. Avoid ornamental scroll reveals, parallax, bounce, pulsing decoration, and simulated typing unless a product requirement specifically earns them.

6.8 Iconography and imagery

Define one icon language with:

- Grid and optical size.
- Stroke or fill treatment.
- Standard sizes.
- Alignment and bounding-box rules.
- Active, inactive, disabled, and inverse behavior.
- Accessible-label policy.
- Rules for badges and notification dots.

Do not mix icon families casually. Icons must clarify an action, object, state, or navigation destination; they must not be added merely to fill space.

For imagery, define cropping, aspect ratios, captions, loading behavior, alt text, attribution, and when generated or stock imagery is prohibited.

6.9 Content and voice

Create a concise content standard covering:

- Sentence case.
- Button labels as direct actions.
- Error messages that state the problem and recovery.
- Empty states that explain what is absent and what can happen next.
- Dates, times, time zones, units, currency, percentages, and ranges.
- Inclusive and nonjudgmental health language.
- Privacy and uncertainty language.
- Confirmation and destructive-action copy.
- Report claims, qualifications, sources, captions, and methodology.

Avoid generic hype, unexplained jargon, internal authoring language, repetitive section labels, and copy that claims more than the evidence supports.

6.10 Accessibility

Target WCAG AA as a minimum and verify the current applicable standard before release. Define:

- Semantic HTML and landmark requirements.
- Heading-order rules.
- Keyboard navigation and focus order.
- Visible focus treatment.
- Pointer target sizes.
- Accessible names and descriptions.
- Error identification and recovery.
- Live-region behavior.
- Screen-reader treatment for charts and dynamic results.
- Reduced motion, high contrast, zoom, and reflow.
- Theme contrast.
- Cognitive accessibility and plain language.
- Non-color cues.

Accessibility requirements are component acceptance criteria, not optional documentation notes.

6.11 Internationalization

Design and test for:

- Text expansion.
- Right-to-left layouts.
- Locale-aware number and date formatting.
- Currency and unit placement.
- Pluralization.
- Mixed writing systems.
- Long unbreakable content.
- Time zones and calendar assumptions.

Avoid fixed widths that work only for English labels. Do not encode direction with left/right naming when start/end semantics are appropriate.

6.12 Data visualization

Define visual encodings before chart components:

- Magnitude and rank use aligned position or length on a common scale.
- Change over time uses horizontal order and aligned position.
- Composition uses proportion only when parts genuinely form a whole.
- Thresholds use distance from a clearly labeled boundary.
- Processes and dependencies use connection and sequence.
- Uncertainty uses intervals, ranges, or explicit annotations.

Require units, periods, populations, bases, missing-data treatment, comparators, and meaningful captions near the evidence they qualify. Prefer direct labels to legends. Provide semantic tables or text alternatives for material data. Never use a chart when a sentence or small table communicates the finding faster.

7. Application mode

Application mode must feel cohesive without forcing every product into the same shell.

7.1 Application composition rules

- Identify the user's primary job for the current screen.
- Establish one dominant region or action rather than giving every panel equal weight.
- Use persistent navigation only when users need repeated movement among peer destinations.
- Group related information through alignment and spacing before adding containers.
- Reserve strong surfaces and brand color for selection, state, hierarchy, or action.
- Keep dashboards task-oriented; do not fill them with metric cards simply because data exists.
- Make dense information scannable through shared grid lines, stable label lanes, tabular numerals, and consistent controls.
- Ensure every interactive component has complete pointer, keyboard, focus, loading, error, empty, disabled, and permission states where applicable.
- Make touch and desktop interactions equally intentional.

7.2 Application shell variants

Document and implement:

- Navigation rail plus top bar.
- Collapsible sidebar plus content header.
- Bottom navigation for compact screens.
- Master-detail layouts.
- Full-screen focused tasks.
- Settings and preference layouts.
- Authentication and onboarding shells.

For each shell define allowed navigation depth, responsive transitions, scroll ownership, sticky behavior, page titles, global actions, local actions, breadcrumbs, and error boundaries.

7.3 Health-product requirements

The health reference must explicitly cover:

- Daily summary and trends.
- Goals and progress.
- Measurements with units and normal/target ranges.
- Manual entry and correction.
- Device synchronization and stale data.
- Missing, partial, estimated, and delayed data.
- Sensitive-data privacy.

- Alerts that avoid unsupported diagnosis or alarmist language.
- Longitudinal charts with accessible summaries.
- Positive, neutral, and concerning states without depending on color.
- User-configurable units and accessibility settings.

8. Report mode

Report mode is a first-class system, not application components placed on printable pages. It must produce clean, confident, evidence-led websites and PDFs comparable in craft to the supplied report reference while remaining recognizably part of the new system.

8.1 Report operating principles

1. Begin with the reader's decision or question, not the report category.
2. Preserve facts, formulas, units, periods, populations, qualifications, uncertainty, privacy constraints, and sources before optimizing presentation.
3. Make the opening view communicate the strongest supported finding and the evidence that earns it.
4. Support two reading speeds: a fast path through titles, headings, decisive values, and captions; and an audit path through tables, assumptions, methods, caveats, and citations.
5. Give each important claim one primary evidence location. Avoid repeating the same conclusion in a hero, card, chart, summary, and conclusion.
6. Choose evidence geometry before choosing a chart or component.
7. Use composition and typography before color, cards, icons, or decoration.
8. Keep exhaustive records available without allowing them to dominate the first read.
9. End with the resolved implication, decision, action, or honest open question.
10. Build trust through clarity and traceability, not exaggerated confidence.

8.2 Report planning canvas

Before designing, record:

- Intended reader and context.
- Decision, question, or understanding required.
- Strongest supported answer.
- Decisive evidence.
- Material caveat that could change interpretation.
- Exact source for every material number.
- Status of each claim: observation, derivation, projection, recommendation, or causal assertion.
- Required privacy, confidentiality, approval, and attribution language.
- What belongs in the executive path.
- What belongs in the audit path.
- What should be omitted because it does not change the decision.

If missing information could change commercial meaning, legal meaning, security claims, privacy, formulas, units, population, time period, recommendation, ownership, approval, deadline, or call to action, pause and request the missing information. Otherwise omit the unknown, label it honestly, and proceed.

8.3 First-view requirements

The first screen or first PDF page must contain enough information for the reader to understand the report's central finding, comparison, decision, or working tool. Do not spend the opening on a ceremonial masthead followed by several paragraphs of setup.

Choose an opening type based on the material:

- Recommendation-led: Answer and decisive basis are co-primary.
- Comparison-led: Alternatives appear on the same basis so the difference can be seen immediately.
- Trend-led: The important relationship, inflection, or exception leads.
- Tool-led: The calculator or interactive model is immediately usable when changing assumptions is the reader's main job.
- State-led: When no decision is justified, lead with the strongest supported state, limit, or unresolved question.

Test the opening by hiding everything after the first screen and asking a reviewer to state what the report decided or established.

8.4 Editorial hierarchy

- Use one descriptive [h1](#) that states the finding or question rather than naming the document type.
- Use sentence-case headings that each answer a new reader question.
- Keep headings, key values, captions, and conclusions sufficient for the fast reading path.
- Keep methods, assumptions, exact tables, notes, and sources sufficient for the audit path.
- Use a controlled prose measure and comfortable line height.
- Inspect line breaks in large headings and rewrite awkward copy before shrinking it.
- Use metadata only for sourced items such as author, recipient, period, status, or confidentiality.
- Do not repeat metadata in a second introductory block.

8.5 Report layout

Define a shared report grid used by identity, title, sections, evidence, tables, figures, and footer. Reading prose should occupy a narrower measure; tables, charts, diagrams, calculators, and major comparisons may use the full evidence width.

Every object must align with a shared edge, baseline, grid line, or deliberate optical center. Repeated values and peer comparisons must share exact tracks. Unequal findings must not be forced into equal cards solely for symmetry.

Open space must strengthen the focal object. Large accidental holes caused by underfilled columns, orphaned items, or delayed evidence must be recomposed rather than defended as minimalism.

8.6 Evidence rules

- Show units, periods, populations, bases, and comparators next to the relevant evidence.
- Use consistent precision and do not create false precision.
- Use zero baselines for length encodings unless a clearly labeled alternative better answers the question.
- Do not compare raw counts when different denominators make rates the relevant measure.
- Do not crop or scale charts in ways that exaggerate differences.
- State selection and filtering rules for subsets.
- Distinguish missing, zero, not applicable, suppressed, and unavailable values.
- Keep the source and method traceable for every important result.
- Pair color with labels, shape, position, line style, or another non-color cue.

8.7 Tables

Tables must use semantic structure and include a useful caption. Define:

- Text columns and their headers align to the start edge.
- Numeric columns and their headers align to the end edge with tabular numerals.
- Comparable columns use the same unit and precision.
- Body cells align by their first text baseline.
- The row-label column receives enough width for ordinary labels.
- Dense ledgers may scroll locally only after reordering and simplification are considered.
- Repeated categories should use groups rather than wasting a column when row-level sorting does not require the repeated value.
- Totals, estimates, exceptions, unavailable values, and confidence ranges have documented treatments.
- A recommendation is highlighted only when the evidence actually supports it.

Tables with many columns should normally own the full evidence width. Do not crush a table beside decorative prose merely to create a split composition.

8.8 Charts and figures

Every chart specification must include:

- The reader question.
- The relationship encoded.
- Why a chart is superior to prose or a table.
- Scale, baseline, domain, units, period, and population.
- Direct labels or a justified legend.
- Primary and supporting series hierarchy.
- Caption describing what to notice and what the chart does not establish.
- Text alternative and accessible data table.
- Narrow-screen behavior.
- Light, dark, print, and monochrome behavior.

Repeated bars must use one shared label lane, plot lane, value lane, annotation lane, and common scale. Only encoded values may change the mark length.

8.9 Calculators and interactive reports

Define a single canonical state model including variables, fixed inputs, formulas, units, precision, ranges, increments, defaults, dependencies, validation, and output formatting.

- One control owns each variable.
- Fixed assumptions are displayed as assumptions, not fake controls.
- Default results are pre-rendered.
- Dependent outputs update together from full-precision state.
- Invalid input remains visible with a useful recovery message; do not silently clamp it.
- Keyboard and screen-reader users can operate the complete model.
- Exported reports record inputs, assumptions, calculation version, timestamp, and result.
- The interface does not duplicate the same default result in a decorative summary above or below the tool.

8.10 Report surfaces and color

Report mode should begin in monochrome. Add color only when it communicates a meaningful state, action, category, series, threshold, or annotation. Do not make a favorable number green simply because it is favorable.

The default report is a continuous canvas. Use a contrasting band, note, rule, or bounded surface only when it establishes a real grouping, warning, selection, tool, or focal statement that spacing and typography cannot communicate sufficiently.

Avoid:

- A generic centered hero followed by cards.
- A card around every metric or section.
- Cards nested inside cards.
- Decorative gradients, glows, glass, textures, grid backgrounds, or ornamental shadows.
- Tiny gray copy used to make dense content fit.
- Pill-shaped metadata and decorative labels.
- Icon tiles and mixed icon families.
- Decorative charts or repeated statistics with no new reader task.
- A dark rounded container around every visualization.
- Repeating the same finding in several equally prominent places.
- Identical section silhouettes throughout a long report.
- Theme controls in a formal delivered report unless the product context requires them.

Restraint must not become sterility. Create presence through strong writing, proportion, line breaks, evidence placement, alignment, density changes, and one decisive relationship at a time.

8.11 PDF and print specification

Report mode must produce polished browser and PDF output.

Define:

- Supported paper sizes, defaulting to A4 with an optional US Letter profile.
- Print margins and page-box behavior.
- Running header and footer rules.
- Page numbering.
- Cover and title-page variants.
- Table-of-contents behavior for long reports.
- Orphan and widow handling.
- `break-before`, `break-after`, and `break-inside` policies.
- Repeated table headers across pages.
- Figure, caption, source, and note cohesion.
- Footnote or endnote system.
- Hyperlink display in print.
- Monochrome and grayscale legibility.
- Embedded fonts and image resolution.
- Tagged-PDF and selectable-text requirements where the export tool supports them.
- Document title, author, subject, language, and creation metadata.

Before delivery, render every PDF page to images and inspect for clipped text, stranded headings, split figures, broken tables, missing glyphs, low-resolution imagery, accidental blank pages, weak page endings, and footer collisions. Also reopen the PDF to verify page count, metadata, selectable text, and link behavior.

9. Component system

Version 1.0 must include a coherent core, not necessarily every possible component.

9.1 Required component families

Layout and structure

- App shell.
- Page container.
- Stack, inline cluster, wrap, split, and grid.
- Section and content group.
- Divider.
- Scroll area.
- Aspect-ratio frame.
- Sticky region.
- Responsive visibility and visually hidden utilities.

Actions

- Button.
- Icon button.
- Floating or prominent action.
- Link and action link.
- Button group.
- Split button when justified.
- Menu button.

Inputs

- Text field.
- Text area.
- Search and autocomplete.
- Checkbox.
- Radio group.
- Switch.
- Select and combobox.
- Slider and range.
- Stepper or quantity input.
- Date and time input.
- File upload.

- Segmented control.
- Form field wrapper, helper, validation, and error summary.

Navigation

- Top application bar.
- Sidebar.
- Navigation rail.
- Navigation drawer.
- Bottom navigation.
- Tabs.
- Breadcrumbs.
- Pagination.
- Step indicator.
- Command or quick-action surface when supported.

Information and surfaces

- Card.
- List and list item.
- Description list.
- Data table.
- Accordion and disclosure.
- Badge and status indicator.
- Avatar.
- Chip or tag, with strict usage criteria.
- Tooltip.
- Popover.
- Menu.
- Dialog.
- Sheet or drawer.
- Metric or statistic.
- Timeline.
- Code block.

Feedback and state

- Inline message.
- Alert or banner.
- Toast or snackbar.
- Progress indicator.
- Skeleton.
- Empty state.
- Error state.
- Offline and stale-data state.

- Confirmation and destructive-action pattern.

Data visualization

- Figure shell.
- Chart header and caption.
- Bar, line, area, dot, distribution, threshold, and comparison primitives where evidence justifies them.
- Legend and direct-label patterns.
- Annotation.
- Accessible data-table companion.
- KPI or metric relationship.

Report primitives

- Report shell.
- Masthead and document metadata.
- Opening claim and opening evidence.
- Reading column.
- Full-width evidence section.
- Stat line or restrained statistic group.
- Comparison structure.
- Method note.
- Formula block.
- Sources and citations.
- Footnote or endnote.
- Contrast field.
- Running footer.

9.2 Component documentation contract

Every component page must contain:

1. Name, status, version introduced, package, and import path.
2. Purpose in one sentence.
3. When to use.
4. When not to use and the preferred alternative.
5. Anatomy with named parts.
6. Variants and sizes.
7. Complete interaction and data states.
8. Behavioral rules.
9. Content rules.
10. Layout, alignment, spacing, and intrinsic sizing.
11. Responsive and container behavior.
12. Keyboard behavior.
13. Focus behavior.
14. Screen-reader semantics and accessible-name requirements.

15. Theme and high-contrast behavior.
16. Localization and long-content behavior.
17. Tokens used.
18. Typed implementation API.
19. Copyable examples.
20. Correct and incorrect examples.
21. Testing requirements.
22. Related components and patterns.
23. Migration and deprecation notes.

Use normative language deliberately:

- MUST: Required for correctness, identity, safety, accessibility, or compatibility.
- SHOULD: Expected default; deviations require a documented reason.
- MAY: Optional behavior that remains inside the system.

9.3 State completeness

For every interactive component, consider:

- Default.
- Hover.
- Focus-visible.
- Pressed or active.
- Selected.
- Disabled.
- Read-only.
- Loading.
- Empty.
- Error.
- Success.
- Pending.
- Dragged or drop target.
- Offline or stale.
- Permission-restricted.
- Long content.
- Narrow container.
- High contrast.
- Reduced motion.

Only document states that make sense for the component, but never omit a state merely because it is visually inconvenient.

10. Patterns and templates

Components do not teach composition. Publish patterns that connect user intent, information architecture, content, components, and responsive behavior.

10.1 Required application patterns

- Dashboard prioritization.
- Metric summary and trend.
- Search, filter, sort, and saved views.
- Create, edit, validate, and confirm forms.
- Multi-step workflows.
- Master-detail navigation.
- Settings and preferences.
- Onboarding and progressive disclosure.
- Authentication and account recovery.
- Notifications and activity history.
- Empty, loading, error, permission, offline, and stale-data experiences.
- Bulk selection and actions.
- Destructive actions and undo.
- Responsive navigation transitions.
- Dense table to compact-screen transformation.
- Help, guidance, and contextual education.

10.2 Required report patterns

- Executive decision brief.
- Comparison and recommendation.
- Benchmark or trend report.
- Research report with methodology.
- Incident or retrospective report.
- Proposal or business case.
- Interactive calculator.
- Audit ledger and appendix.
- Source and citation system.
- Web-to-PDF export.

10.3 Pattern page contract

Every pattern page must include:

- Reader or user problem.
- Preconditions and required information.
- Content hierarchy.
- Recommended composition.

-
- Components and primitives used.
 - Responsive transformations.
 - Accessibility requirements.
 - Difficult states and edge cases.
 - Full working example.
 - Anti-patterns.
 - Verification checklist.
-

11. Extension protocol

The system must allow an agent to create an app-specific component without giving it permission to invent a parallel design language.

Publish this protocol prominently:

Creating an undocumented component

1. Search the component and pattern indexes for an existing solution.
2. State the user problem and why existing components cannot solve it through composition.
3. Build from existing primitives before creating a new primitive.
4. Use semantic tokens only. Raw values require a documented exception.
5. Inherit the nearest component's typography, shape, density, focus, state, and motion behavior.
6. Define anatomy, states, accessibility, responsive behavior, and content rules before polishing.
7. Test the component in light, dark, narrow, wide, keyboard, zoom, reduced-motion, and long-content contexts.
8. Use a page- or product-owned namespace until the component is accepted into the system.
9. If the solution appears in three independent products, propose system adoption with evidence.
10. Record the extension and its rationale so later agents do not create a conflicting version.

Explicitly prohibit:

- New raw color values.
 - New arbitrary spacing or radius values.
 - Private copies of core components with small visual changes.
 - Unnamed one-off typography roles.
 - A new icon family.
 - Undocumented focus or keyboard behavior.
 - App-specific CSS that targets internal selectors of system components.
-

12. Tokens and machine-readable contracts

Publish tokens as versioned, machine-readable JSON and generated platform outputs.

12.1 Token hierarchy

Use this dependency direction:

reference values -> semantic roles -> component aliases -> rendered component

Example:

```
{
  "color": {
    "reference": {
      "violet": {
        "500": { "$type": "color", "$value": "#6D5CE7" }
      }
    },
    "semantic": {
      "action": {
        "primary": {
          "$type": "color",
          "$value": "{color.reference.violet.500}"
        }
      }
    }
  },
  "space": {
    "4": { "$type": "dimension", "$value": "1rem" }
  },
  "radius": {
    "surface": { "$type": "dimension", "$value": "1.5rem" }
  }
}
```

Generated CSS should expose semantic names:

```
:root {
  --ds-color-canvas: ...;
  --ds-color-surface: ...;
  --ds-color-text-primary: ...;
  --ds-color-action-primary: ...;
  --ds-space-4: ...;
  --ds-radius-surface: ...;
}
```

Replace the temporary `ds` prefix after naming. Never make product code depend directly on palette values when a semantic role exists.

12.2 Required token outputs

- Canonical token JSON.
- JSON Schema for validation.
- Light and dark theme outputs.
- CSS custom properties.
- TypeScript token typings.
- Optional native-platform transforms when those platforms are adopted.
- Human-readable token reference pages.
- Version and checksum metadata.
- Automated validation for missing references, cycles, invalid types, and undocumented tokens.

13. Implementation packages

Use a monorepo only if it improves maintenance in the chosen environment. A recommended structure is:

```
apps/  
  docs/  
    reference-health/  
    reference-finance/  
    reference-home/  
    reference-report/  
packages/  
  tokens/  
  icons/  
  foundations/  
  react/  
  report/  
  eslint-plugin/  
  test-utils/  
docs-content/  
  foundations/  
  components/  
  patterns/  
  references/  
agent/  
  llms.txt  
  llms-full.txt  
  manifest.json  
  instructions.md  
  prompts/  
tests/  
  accessibility/  
  visual/  
  agent-comprehension/
```

13.1 Package expectations

Tokens package

- Framework independent.
- Versioned semantic output.
- No undocumented values.
- Tree-shakable or statically consumable where relevant.

Icon package

- One visual family.
- Accessible defaults.
- Stable names and aliases.
- Size and stroke consistency tests.

React reference package

- Typed public APIs.
- Semantic DOM.
- Composable parts without exposing unstable internals.
- Controlled and uncontrolled behavior where justified.
- Ref forwarding and form compatibility.
- Theme, direction, localization, and accessibility support.
- Minimal dependencies with documented reasons.

Report package

- Report shell and grid.
- Editorial typography.
- Evidence, table, figure, citation, note, calculator, and print primitives.
- Stable browser-to-PDF behavior.
- No requirement to use application cards or navigation chrome.

Linting and validation

Where practical, provide rules that detect:

- Raw colors.
- Raw spacing and radii outside documented exceptions.
- Unsupported component imports.
- Invalid token references.
- Missing accessible names.
- Heading-order problems.
- Undocumented or deprecated APIs.

14. Documentation website

The canonical site is <https://design.oscarkalid.com>. It must be useful to designers, engineers, writers, and agents.

14.1 Information architecture

```
/
/start
/principles
/foundations
  /color
  /typography
  /spacing
  /shape
  /elevation
  /layout
  /motion
  /icons
  /content
  /accessibility
  /internationalization
  /data-visualization
/components
  /[component]
/patterns
  /applications/[pattern]
  /reports/[pattern]
/templates
/examples
  /health
  /finance
  /smart-home
  /productivity
  /reports
/tokens
/resources
/changelog
/releases/[version]
/llms.txt
/llms-full.txt
/manifest.json
```

14.2 Documentation-page requirements

- Stable canonical URL.
- Server-rendered or static primary content.
- Equivalent raw Markdown representation.
- Clear title, summary, status, version, and last-reviewed date.
- Deep links for every major section.
- Links to related foundations, components, patterns, code, and examples.
- Copyable code with visible language and package version.
- Good and bad examples with explanations.
- Token names and downloadable values.
- Accessibility and responsive behavior.
- Searchable text without requiring a signed-in session.
- Machine-readable metadata.
- No important guidance hidden only inside video, canvas, image, or client-side interaction.

14.3 Technical accessibility for agents

- Serve `llms.txt` as plain text or Markdown from the domain root.
- Make links absolute and canonical.
- Keep documents readable without JavaScript execution.
- Provide Markdown or text endpoints with predictable URLs.
- Avoid authentication for public design guidance.
- Do not block intended documentation crawlers in `robots.txt`.
- Publish a sitemap and a compact manifest.
- Version breaking documentation changes.
- Preserve old release URLs.
- Keep page content focused so an agent can retrieve only the relevant specification.
- Provide a downloadable offline snapshot for agents without network access.

15. Agent interface

`llms.txt` is an index and routing layer, not the complete system. The actual reliability comes from precise specifications, machine-readable tokens, tested packages, examples, and clear fallback rules.

15.1 Required `llms.txt`

Publish a concise file with this structure:

```
# [SYSTEM_NAME] Design System by Oscar Kalid

> [One-sentence identity and purpose.]

Current stable version: [VERSION]
Canonical documentation: https://design.oscarkalid.com
This system is independent and is not affiliated with Google, Vercel, or OpenAI.

## Start here
```

```

- [Getting started](https://design.oscarkalid.com/start.md)
- [Principles](https://design.oscarkalid.com/principles.md)
- [Tokens](https://design.oscarkalid.com/tokens/tokens.json)
- [Accessibility](https://design.oscarkalid.com/foundations/accessibility.md)
- [Extension protocol](https://design.oscarkalid.com/extensions.md)

## Choose an output mode

- [Application mode](https://design.oscarkalid.com/modes/application.md)
- [Report mode](https://design.oscarkalid.com/modes/report.md)

## Indexes

- [Foundations](https://design.oscarkalid.com/foundations/index.md)
- [Components](https://design.oscarkalid.com/components/index.md)
- [Application patterns](https://design.oscarkalid.com/patterns/applications/index.md)
- [Report patterns](https://design.oscarkalid.com/patterns/reports/index.md)
- [Reference implementations](https://design.oscarkalid.com/examples/index.md)

## Packages

- Tokens: @[scope]/tokens
- Icons: @[scope]/icons
- React: @[scope]/react
- Reports: @[scope]/report

## Mandatory agent rules

- Read the start page, selected mode, relevant foundations, components, and patterns before implementation.
- Use published packages and semantic tokens when available.
- Do not invent raw visual values when a token exists.
- Do not substitute another design system.
- Use the extension protocol for undocumented components.
- Implement accessibility, responsive behavior, themes, and complete states.
- Render and visually inspect the result before delivery.

```

15.2 llms-full.txt

Generate a versioned, concatenated text edition for agents that cannot browse multiple pages. It must:

- Include a generated table of contents.
- Preserve source-page URLs and headings.
- State generation time and design-system version.
- Exclude heavy duplicate examples where a canonical link is sufficient.
- Stay synchronized automatically with documentation releases.
- Never become the only source of truth.

15.3 Machine manifest

Publish `manifest.json` containing:

- System name, version, release date, and canonical URL.
- Supported modes, frameworks, themes, locales, and accessibility target.
- URLs for indexes, tokens, schemas, packages, changelog, and offline bundle.
- Component identifiers, status, version introduced, aliases, and documentation URL.
- Pattern identifiers and applicable modes.
- Deprecated items and replacements.
- Content hash or checksum for key machine-readable artifacts.

15.4 Repository adapters

Provide small templates for:

- [AGENTS.md](#)
- [CLAUDE.md](#)
- [.github/copilot-instructions.md](#)
- Cursor rules or an equivalent local-instruction format.
- A vendor-neutral [DESIGN_SYSTEM.md](#).

These files must not duplicate the entire design system. They should identify the selected version, link to the canonical entry point, identify installed packages, state required verification commands, and point to a local offline snapshot.

Example:

```
## Design system
```

All interface and report work must use [SYSTEM_NAME] [VERSION].

Canonical entry point:

<https://design.oscarkalid.com/llms.txt>

Before implementation, read the selected output mode and the relevant component and pattern specifications. Prefer installed @[scope] packages. Do not introduce arbitrary visual values. Use the extension protocol for app-specific components. Run the documented accessibility, responsive, and visual checks before delivery.

If network access is unavailable, use docs/design-system/[VERSION]/.

15.5 Optional agent tools

After the documentation is stable, consider an MCP server or small API exposing:

- `search_spec(query, mode, version)`
- `get_component(name, framework, version)`
- `get_pattern(name, mode, version)`
- `get_tokens(theme, platform, version)`
- `get_example(category, viewport, theme)`
- `validate_usage(files, version)`

This is an enhancement, not a substitute for public, readable documentation.

16. Reusable prompts

16.1 Application prompt

Build [APPLICATION] using [SYSTEM_NAME] Design System version [VERSION].

Begin at <https://design.oscarkalid.com/llms.txt>. Read the application-mode guide, the relevant foundations, and the applicable component and pattern specifications before coding. Use the official token and component packages. Do not substitute Material 3, shadcn defaults, Tailwind defaults, or another visual system.

You may create domain-specific components only through the published extension protocol. Implement narrow and wide layouts, light and dark themes, keyboard and screen-reader behavior, loading, empty, error, disabled, and long-content states. Use realistic content. Render the result at the documented viewports, inspect it visually, run accessibility and build checks, and correct material defects before delivery.

16.2 Report prompt

Create a [REPORT TYPE] using [SYSTEM_NAME] Report Mode version [VERSION].

Begin at <https://design.oscarkalid.com/llms.txt> and read the report-mode, editorial hierarchy, evidence, table, chart, citation, and PDF specifications. Inspect all supplied source material before designing.

Preserve every material fact, formula, unit, period, population, qualification, uncertainty, privacy constraint, and source. Identify the reader's question, the strongest supported answer, the evidence that earns it, and the caveat that could change it. Make the first view communicate that central relationship. Support a fast executive reading path and a complete audit path without repeating the same claim at equal prominence.

Choose evidence geometry before components. Use color and surfaces only when they carry meaning. Produce a responsive web version and a polished accessible PDF. Render every PDF page and visually inspect tables, figures, page breaks, type, headers, footers, and sources before delivery.

16.3 Audit prompt

Audit this artifact against [SYSTEM_NAME] version [VERSION]. Do not redesign it yet. Identify violations in descending order of effect on user comprehension, task completion, accessibility, responsive behavior, evidence integrity, system consistency, and visual quality. Cite the governing system rule for every finding. Separate objective violations from judgment calls. Recommend the smallest coherent set of corrections and state which checks will verify them.

17. Testing and quality assurance

The release must be tested as a design system, implementation library, documentation product, and agent interface.

17.1 Automated testing

- Token schema validation.
- Token reference and cycle validation.
- Component unit and interaction tests.
- Keyboard behavior tests.
- Automated accessibility checks.
- Theme snapshots.
- Right-to-left snapshots.
- Reduced-motion behavior.
- High-contrast or forced-colors behavior where supported.
- Visual regression at documented viewports.
- Documentation code-example compilation.
- Broken-link and anchor checks.
- Manifest and [llms.txt](#) link checks.
- Package build, type, lint, and export tests.
- PDF generation and page-count checks.

17.2 Manual visual review

Review application references at a minimum of:

- Narrow phone.
- Large phone.
- Tablet or compact desktop.
- Standard desktop.
- Wide desktop.
- 200 percent zoom.

Review reports as:

- Responsive browser page.
- Narrow browser page.
- Light theme.
- Dark theme where supported.
- A4 PDF.
- US Letter PDF if supported.
- Grayscale or monochrome print.

Inspect:

- First-read hierarchy.
- Stable reading order.
- Shared alignment lines.
- Relational spacing.
- Type roles and line breaks.
- Meaningful use of color and surfaces.
- Complete states.
- Long labels and extreme values.
- Focus visibility and keyboard order.
- Chart scales and labels.
- Table alignment and wrapping.
- Empty and error states.
- Page breaks, captions, footers, and sources.

17.3 Agent-comprehension tests

Use at least three independent agent runs with clean context. Do not provide hidden design hints beyond the public O3 entry point and the task brief.

Required tasks:

1. Build a health-tracker screen using documented components.
2. Build an application category absent from the reference gallery using the extension protocol.
3. Produce a short evidence-led report with a table, chart, caveat, citations, and PDF export.

Score each run on:

- Correct documentation discovery.
- Correct mode selection.
- Published component usage.
- Semantic token usage.

- Absence of arbitrary visual values.
- Accessibility.
- Responsive behavior.
- State completeness.
- Visual identity.
- Composition quality.
- Report evidence integrity where applicable.
- Successful build and export.

After each run, classify every failure as one of:

- Documentation missing.
- Documentation ambiguous.
- Retrieval or routing failure.
- Package/API limitation.
- Example bias.
- Agent ignored explicit guidance.
- Visual judgment gap.

Improve the system for systemic failures. Do not patch the evaluation prompt with secret instructions that real users will not have.

17.4 Acceptance thresholds

Version 1.0 cannot ship with:

- Broken required links.
- Invalid token references.
- Critical automated accessibility violations in reference artifacts.
- Keyboard traps.
- Unreadable content at supported widths or zoom.
- Undocumented public components.
- Examples that rely on private or arbitrary visual values.
- Reports with clipped content, broken tables, missing sources, or misleading evidence geometry.
- Agent instructions that require inaccessible private context.

Every reference artifact must pass build, type, lint, accessibility, responsive, and visual review. Known lower-severity limitations must be documented with owners and intended resolution.

18. Governance and release management

18.1 Decision records

Maintain short architectural and design decision records for material choices such as:

- Naming.
- Base typefaces.
- Color model and contrast approach.

- Token naming.
- Component API conventions.
- The relationship between application and report modes.
- Framework adoption.
- Browser support.
- Breaking accessibility or behavior changes.

Each record should state context, decision, alternatives, consequences, and date.

18.2 Versioning

Use semantic versioning for code packages and a synchronized documentation release identifier.

- Patch: corrections that do not intentionally change public behavior.
- Minor: additive tokens, components, variants, or patterns.
- Major: renamed or removed APIs, token meaning changes, or behavior changes requiring consumer work.

Publish:

- Changelog.
- Migration guides.
- Deprecation warnings.
- Support window.
- Release notes written for both humans and agents.
- Version-pinned documentation URLs.

Do not silently change the meaning of an existing semantic token.

18.3 Contribution process

Every proposed addition must include:

- User problem.
 - Evidence of recurrence.
 - Existing alternatives considered.
 - Design and content specification.
 - Accessibility review.
 - Token impact.
 - API and implementation.
 - Examples and anti-examples.
 - Tests.
 - Documentation.
 - Migration or compatibility impact.
-

19. Execution plan

Phase 0 - Establish the project

- Inspect the repository and source assets.
- Confirm the owner, canonical domain, intended platforms, and publishing environment from available context.
- Create the decision log, issue structure, and release checklist.
- Run the naming process and replace [SYSTEM_NAME] once selected.
- Record explicit assumptions rather than blocking on low-risk choices.

Exit: Stable name or documented working-name decision, repository structure, scope, domain, and evaluation plan.

Phase 1 - Define visual judgment

- Analyze references.
- Write the charter and personality scales.
- Create two or three materially different visual directions.
- Test directions on one application screen and one report page.
- Select and document the strongest direction.
- Define signature moves and rejected defaults.

Exit: Approved or explicitly selected direction that works in both modes.

Phase 2 - Build foundations

- Implement token architecture.
- Complete color, type, spacing, shape, elevation, grid, motion, icons, content, accessibility, internationalization, and data-visualization rules.
- Generate theme outputs and token documentation.
- Create foundation tests.

Exit: Machine-readable and documented foundations with passing validation.

Phase 3 - Build the core system

- Implement primitives and high-priority components.
- Write complete component contracts.
- Create examples and state matrices.
- Add automated tests and visual stories.
- Implement report primitives independently from application surface habits.

Exit: Core components are usable, accessible, documented, and versioned.

Phase 4 - Prove composition

- Build the five application references.
- Build the three report references.
- Extract and document recurring patterns.
- Test difficult content and responsive transformations.
- Export and visually verify reference PDFs.

Exit: The system produces convincing full artifacts, not only isolated components.

Phase 5 - Publish the agent layer

- Build the documentation site.
- Publish Markdown pages, `llms.txt`, `llms-full.txt`, tokens, schema, manifest, and offline bundle.
- Add repository instruction templates and reusable prompts.
- Verify unauthenticated, non-JavaScript access to primary guidance.
- Add versioning and link tests.

Exit: A clean-context agent can discover and retrieve the correct guidance.

Phase 6 - Validate and release

- Run independent agent-comprehension tests.
- Fix systemic documentation, API, token, and visual issues.
- Perform manual accessibility and visual review.
- Publish release notes, known limitations, and migration expectations.
- Deploy to design.oscarkalid.com.

Exit: All version 1.0 criteria pass and required artifacts are publicly available.

20. Required deliverables

The project is not complete until it contains:

Identity and governance

- ☐ Final name and identity statement.
- ☐ Canonical domain and redirect plan.
- ☐ Design charter and signature moves.
- ☐ Decision records.
- ☐ Changelog and release policy.

Foundations

- ☐ Color and themes.
- ☐ Typography.
- ☐ Spacing and density.
- ☐ Shape.
- ☐ Surfaces and elevation.
- ☐ Layout and responsive behavior.
- ☐ Motion and reduced motion.
- ☐ Icons and imagery.
- ☐ Content and voice.
- ☐ Accessibility.

- ☐ Internationalization.
- ☐ Data visualization.

Implementation

- ☐ Token JSON and schema.
- ☐ Generated CSS and TypeScript output.
- ☐ Icon package.
- ☐ Core component package.
- ☐ Report package.
- ☐ Linting or validation rules.
- ☐ Tests and visual references.

Documentation

- ☐ Documentation website.
- ☐ Complete component pages.
- ☐ Application patterns.
- ☐ Report patterns.
- ☐ Extension protocol.
- ☐ Five application references.
- ☐ Three report references.
- ☐ PDF and print guidance.
- ☐ Versioned changelog and migration pages.

Agent interface

- ☐ `/llms.txt`.
- ☐ `/llms-full.txt`.
- ☐ `/manifest.json`.
- ☐ Raw Markdown pages.
- ☐ Offline documentation bundle.
- ☐ `AGENTS.md` template.
- ☐ `CLAUDE.md` template.
- ☐ Copilot/Cursor or equivalent adapters.
- ☐ Reusable application, report, and audit prompts.
- ☐ Clean-context agent evaluation suite.

Quality

- ☐ Automated accessibility checks.
- ☐ Manual keyboard and screen-reader review of core flows.
- ☐ Responsive and zoom review.
- ☐ Light, dark, high-contrast, and reduced-motion review.
- ☐ Visual regression coverage.

- ☐ PDF rendering and inspection.
- ☐ Broken-link and code-example verification.
- ☐ Version 1.0 acceptance record.

21. Definition of done

The design system is ready for version 1.0 only when all of the following are true:

1. A new agent can find the correct entry point from the canonical domain.
2. The agent can identify whether a task requires application mode or report mode.
3. The agent can find relevant foundations, components, patterns, tokens, and examples without reading the entire site.
4. The agent can build an unfamiliar application without inventing a second visual language.
5. The agent can create a domain-specific component using the extension protocol.
6. The agent can produce a report whose argument, evidence, tables, charts, citations, responsive layout, and PDF output meet the documented quality bar.
7. Reference implementations use the published system rather than private values or undocumented code.
8. Core components meet accessibility, keyboard, theme, responsive, localization, and state requirements.
9. Documentation and packages are synchronized and versioned.
10. Independent agent runs produce related artifacts without collapsing into one repetitive template.
11. The result has a recognizable identity that is not merely a recolored copy of another design system.
12. No known material defect remains in the release artifacts.

22. First actions for the executing agent

Begin immediately with the following sequence:

1. Inspect the current repository and list relevant assets, frameworks, and constraints.
2. Create a project status document mapping existing material to the required deliverables.
3. Review the supplied application screenshot and report PDF for underlying qualities, not copyable decoration.
4. Create the design-system decision log.
5. Run the naming and identity phase while keeping design.oscarkalid.com as the canonical domain.
6. Draft the design charter, mode distinction, and signature-move hypotheses.
7. Create two materially different visual directions applied to one health-dashboard screen and one report opening.
8. Select the direction that best satisfies clarity, originality, composability, accessibility, and cross-mode coherence.
9. Implement foundations and tokens before expanding the component inventory.
10. Continue through the remaining phases and maintain the release checklist until the definition of done is satisfied.

When a missing choice does not create material risk, make the most defensible assumption, record it, and continue. When a missing fact would change legal, privacy, security, health, commercial, or evidence claims, request that fact explicitly rather than inventing it.

Deliver completed artifacts and verification results. Do not finish by returning only another plan.

Appendix A – Fast visual inspection routine

Use this order when reviewing any generated artifact:

1. Purpose: Can a new viewer state the screen's task or report's finding after the first view?
2. Hierarchy: Is one relationship dominant, or does every block compete equally?
3. Structure: Do shared edges, baselines, and tracks make groups and comparisons obvious?
4. Typography: Are equivalent roles consistent and body text comfortably readable?
5. Spacing: Does every gap express a relationship and have one owner?
6. Color: Does every strong color carry meaning, state, or identity?
7. Surfaces: Can a box, border, or shadow be removed without losing grouping or affordance?
8. Interaction: Are focus, keyboard, loading, errors, and destructive actions complete?
9. Evidence: Are units, bases, scales, sources, and caveats visible where needed?
10. Adaptation: Does the composition genuinely reflow at narrow widths and high zoom?
11. Identity: Does the result feel recognizably part of the system without repeating a template?
12. Restraint: Remove anything that adds visual activity but no meaning, usability, or rhythm.

Fix the highest-impact systemic issue first, rerender, and repeat.

Appendix B – Component specification template

```
---
id: [component-id]
name: [Component name]
status: draft | beta | stable | deprecated
version_introduced: [version]
package: [package]
mode: shared | application | report
last_reviewed: [date]
---

# [Component name]

> [One-sentence purpose.]

## Use when
## Do not use when
## Anatomy
## Variants
## Sizes and density
## States
## Behavior
## Content
## Layout and intrinsic sizing
## Responsive behavior
## Keyboard interaction
## Focus management
## Accessibility semantics
## Themes and high contrast
## Localization and long content
## Tokens
## Typed API
## Examples
## Incorrect examples
## Testing requirements
## Related components and patterns
## Migration and deprecation
```

Appendix C – Pattern specification template

```
---
id: [pattern-id]
name: [Pattern name]
status: draft | beta | stable | deprecated
mode: application | report
version_introduced: [version]
last_reviewed: [date]
---

# [Pattern name]

## User or reader problem
## Preconditions and required information
## Primary task or question
## Information hierarchy
## Recommended composition
## Components and primitives
## Content guidance
## Responsive transformations
## Accessibility requirements
## Loading, empty, error, and edge states
## Complete example
## Anti-patterns
## Verification checklist
```

Appendix D – Report preflight

Before delivering a report, verify:

- ☐ Every material claim is supported by a supplied or cited source.
- ☐ Facts are distinguished from derivations, projections, recommendations, and causation.
- ☐ Units, periods, populations, denominators, and qualifiers are preserved.
- ☐ The first view communicates the central finding or working tool.
- ☐ Headings, decisive values, and captions form a coherent fast reading path.
- ☐ Methods, assumptions, caveats, tables, and sources form a complete audit path.
- ☐ Each major claim has one primary evidence location.
- ☐ Every visualization uses geometry suited to the relationship.
- ☐ Tables use correct semantic structure and alignment.
- ☐ Color is meaningful and has a non-color counterpart.
- ☐ The report remains legible in monochrome.
- ☐ Narrow layout and zoom do not clip or reorder meaningfully related content incorrectly.
- ☐ PDF pages have been rendered and visually inspected.
- ☐ Metadata, page numbering, links, selectable text, and fonts have been checked.

Appendix E - Source and inspiration notes

- Vercel's public report-design guidance: <https://vercel.com/design.md>.
- User-supplied report-quality reference: [What decides a Vercel report.pdf](#).
- Material 3: completeness and ecosystem-quality benchmark, not a visual-copying mandate.
- User-supplied dark dashboard screenshot: application coherence reference, not a fixed O3 specification.

The final system must translate the valued qualities from these references into original principles, tokens, components, patterns, and verification rules owned by Oscar Kalid.